

LABORATORY

LAB 14

DC Motor Control

Speed and Direction Control with a Motor Driver Module

S32K144 · MaaZEDU

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	DC Motor	2
2.2	H-Bridge Motor Driver Module	3
2.3	Speed Control with PWM	3
3	Hardware Setup	4
3.1	Required Hardware	4
3.2	Control Pin Mapping	4
3.3	Logic Supply and Motor Supply	4
3.4	Connection Diagram	5
4	EduFramework API	5
4.1	DCMotor_Init()	5
4.2	DCMotor_SetSpeed()	6
4.3	DCMotor_Forward()	6
4.4	DCMotor_Reverse()	7
4.5	DCMotor_Stop()	7
5	Lab Exercise	7
5.1	Requirements	7
5.2	Program	7
5.3	Program Explanation	8
5.4	Verification	8
6	Extension	9
6.1	Changing the Speed Level	9
6.2	Coast Stop and Short Brake	9
6.3	Standby Mode	10
7	References	10

1 Introduction

1.1 Lab Overview

PWM has been used in previous labs to control the output level applied to a load. For power loads such as DC motors, a microcontroller does not normally supply the motor current directly. Instead, a motor driver module is used as an intermediate power stage between the control signals and the load.

H-bridge motor driver modules commonly provide three basic functions: changing the direction of rotation, controlling the applied level using PWM, and placing the motor in a stopped state. In this lab, the TB6612FNG is used as a typical example of this type of module. The TB6612FNG supports bidirectional control, PWM operation, stop, short brake, and standby [3].

At the application level, EduFramework provides a DC Motor Device API that allows the program to configure the motor control level, direction, and stop state without directly manipulating each control signal of the H-bridge.

The lab exercise focuses on a simple operating sequence: the motor rotates forward at approximately 50% control level, stops, rotates in reverse at the same level, and continuously repeats the sequence.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the role of a motor driver module in a DC motor control system.
2. Explain the basic principle of an H-bridge for direction control and the role of PWM in controlling the applied motor level.
3. Use the `DCMotor_Init()`, `DCMotor_SetSpeed()`, `DCMotor_Forward()`, `DCMotor_Reverse()`, and `DCMotor_Stop()` APIs.
4. Build and verify a motor control program that follows the sequence forward → stop → reverse → stop.

2 Background

2.1 DC Motor

A DC motor is an actuator that converts direct-current electrical energy into rotational motion. In a basic control system, reversing the polarity applied across the motor terminals changes the current direction through the motor and therefore changes its direction of rotation. The application therefore requires a power stage capable of controlling current in both directions rather than a single digital output [1].

For open-loop speed control, PWM can be used to change the average power applied to the motor. The actual mechanical speed depends not only on the PWM value but also on the motor characteristics, supply voltage, and mechanical load [2].

2.2 H-Bridge Motor Driver Module

An H-bridge is a structure commonly used to control the direction of a DC motor. By changing the states of the power switches, the polarity applied across the motor terminals can be reversed to produce two directions of rotation [1].

An H-bridge motor driver module commonly provides the following groups of signals:

- direction control signals;
- a PWM signal for controlling the applied level;
- an enable or standby signal, depending on the module.

Pin names, logic levels, and signal organization can differ between modules. Therefore, when replacing the TB6612FNG with another motor driver module, its technical documentation must be checked before making the connections.

In the hardware configuration used in this lab, the TB6612FNG serves as a typical example. Channel A of the module uses four main control signals:

- **AIN1** and **AIN2** : select the operating state and direction;
- **PWMA** : PWM input for channel A;
- **STBY** : enables the module or places it in standby mode.

When **STBY** is active and **PWMA** enables the output, the basic relationship between **AIN1** , **AIN2** , and the channel A state can be summarized as follows [3] :

AIN1	AIN2	State
HIGH	LOW	Rotation in one direction.
LOW	HIGH	Rotation in the opposite direction.
LOW	LOW	Stop with the output in a high-impedance state.
HIGH	HIGH	Short brake.

The `DCMotor_Forward()` and `DCMotor_Reverse()` APIs represent two opposite control configurations. The actual physical direction of rotation still depends on how the motor terminals are connected to **A01** and **A02**.

2.3 Speed Control with PWM

The principles of PWM, period, and duty cycle were introduced in Lab 05 and are therefore not repeated in detail in this lab. In motor control, PWM is applied to the control input of the driver module to change the average level applied to the load. PWM and an H-bridge are commonly combined to control the speed and direction of a DC motor [1] [2].

The EduFramework DC Motor Device API uses a control range from **0** to **255** [6] :

Value	Control Level	Meaning
0	0%	No speed control level is applied.
64	approximately 25%	Low control level.
128	approximately 50%	Mid-range control level.
191	approximately 75%	High control level.
255	100%	Maximum control level.

These percentages describe command values within the control range, not the actual motor rotational speed.

3 Hardware Setup

3.1 Required Hardware

The lab uses the MaaZEDU Development Board to generate control signals, an H-bridge motor driver module as the power stage, and a DC motor as the load. An external power supply is used to provide power for the motor.

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
TB6612FNG Motor Driver Module	H-bridge module used as an example for controlling the speed and direction of a DC motor.
DC Motor	Direct-current motor used as the load in the lab exercise.
External Motor Power Supply	Separate power source connected to the VM supply of the motor driver.
Jumper Wires	Connect the power and control signals between the hardware blocks.
USB Cable	Connects MaaZEDU to the computer for logic power and program uploading.

Table 14.3.1. Hardware used in this lab

3.2 Control Pin Mapping

The current EduFramework DC Motor API uses four signals: one PWM signal, two direction control signals, and one standby signal. In the example TB6612FNG configuration, GPIO2 through GPIO5 are connected to PWMA, AIN1, AIN2, and STBY, respectively.

According to the MaaZEDU Development Board Guide, these Logical Pins are mapped to PTD14, PTD13, PTD12, and PTD11, respectively [5].

Component	Logical Pin	MCU Pin	Function
PWMA	GPIO2	PTD14	PWM Output
AIN1	GPIO3	PTD13	Digital Output
AIN2	GPIO4	PTD12	Digital Output
STBY	GPIO5	PTD11	Digital Output

Table 14.3.2. Pin mapping for the example TB6612FNG configuration

Note

If a different motor driver module is used, identify pins with equivalent functions and verify the required logic levels from the module documentation. Do not assume that every module uses the same pin names or operating-state table as the TB6612FNG.

3.3 Logic Supply and Motor Supply

Some motor driver modules use separate logic and power supplies. For the TB6612FNG, VCC powers the control logic, while VM supplies the motor power stage. The datasheet specifies an operating range of 2.7 V to 5.5 V for VCC and 2.5 V to 13.5 V for VM [3].

In the example configuration used in this lab:

- the TB6612FNG VCC pin is connected to 3.3V from MaaZEDU;
- VM is connected to an external power supply suitable for the motor;
- the GND terminals of MaaZEDU, the TB6612FNG, and the motor power supply are connected together;
- AO1 and AO2 are connected to the two terminals of the DC motor.

Note

The external supply voltage must match the rated voltage of the motor and remain within the allowed range of the motor driver module. Turn off the motor power supply before changing hardware connections.

3.4 Connection Diagram

The following diagram illustrates the use of a TB6612FNG as the motor driver module in this lab. MaaZEDU generates the control signals, the TB6612FNG provides the power stage, and the external supply provides energy to the motor.

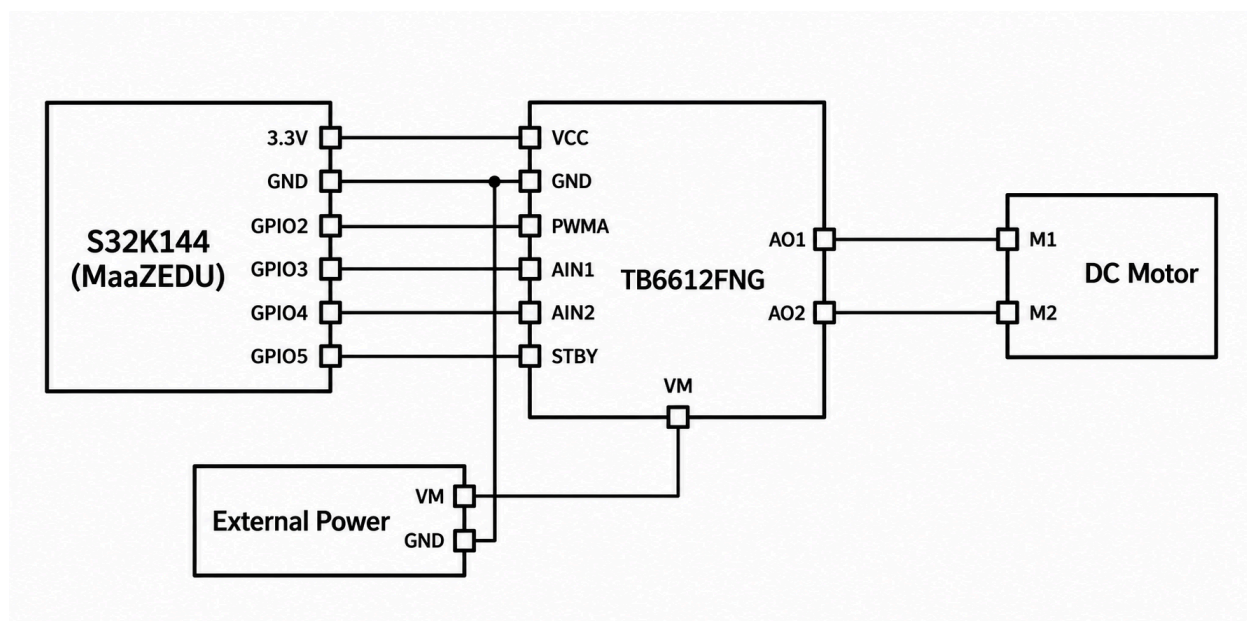


Figure 14.3.1. Example connection diagram using MaaZEDU, TB6612FNG, and a DC motor

4 EduFramework API

This lab uses the EduFramework DC Motor Device API to control the motor at the application level. The PWM and Digital Output operations required by the motor driver module are handled inside the device layer, so the application does not need to manipulate each control pin directly [6].

The current API is based on an interface with one PWM pin, two direction control pins, and one standby pin.

4.1 DCMotor_Init()

`DCMotor_Init()` initializes the pins required by the motor driver module. After initialization, the motor is in a stopped state and the module is enabled.

DCMotor_Init

Initialize the interface used to control one DC motor through a motor driver module.

Syntax

```
DCMotor_Init(pwmPin, in1Pin, in2Pin, stbyPin);
```

Parameters

Parameter	Type / accepted value	Description
pwmPin	PWM-capable Logical Pin	Pin connected to the PWM input of the motor driver module.
in1Pin	Logical Pin	Pin connected to the first direction-control input.
in2Pin	Logical Pin	Pin connected to the second direction-control input.
stbyPin	Logical Pin	Pin connected to the standby input of the motor driver module.

Return value

No return value.

Example with the TB6612FNG:

```
DCMotor_Init(GPIO2, GPIO3, GPIO4, GPIO5);
```

4.2 DCMotor_SetSpeed()

`DCMotor_SetSpeed()` sets the speed control level within the range `0` to `255`. The function changes only the PWM level and does not change the current direction.

DCMotor_SetSpeed

Set the motor speed control level.

Syntax

```
DCMotor_SetSpeed(speed);
```

Parameters

Parameter	Type / accepted value	Description
speed	<code>0</code> to <code>255</code>	Motor speed control value. <code>0</code> is the minimum value and <code>255</code> is the maximum value of the control range.

Return value

No return value.

Example:

```
DCMotor_SetSpeed(128U);
```

The value `128` corresponds to approximately `50%` of the control range.

4.3 DCMotor_Forward()

`DCMotor_Forward()` places the driver module in the forward configuration and applies the previously configured speed level.

DCMotor_Forward

Rotate the motor in the forward direction.

Syntax

```
DCMotor_Forward();
```

Return value

No return value.

The actual physical direction depends on how the two motor terminals are connected to the outputs of the motor driver module.

4.4 DCMotor_Reverse()

`DCMotor_Reverse()` places the module in the opposite configuration to forward rotation.

DCMotor_Reverse

Rotate the motor in the reverse direction.

Syntax

```
DCMotor_Reverse();
```

Return value

No return value.

4.5 DCMotor_Stop()

`DCMotor_Stop()` sets the PWM control level to `0` and places the direction control pins in the stop state. With the example TB6612FNG configuration, the motor is stopped using the coast state [3] [6] .

DCMotor_Stop

Stop the motor using coast mode.

Syntax

```
DCMotor_Stop();
```

Return value

No return value.

5 Lab Exercise

5.1 Requirements

Build a program that controls a DC motor through a motor driver module.

- Initialize the control interface with `GPIO2` , `GPIO3` , `GPIO4` , and `GPIO5` .
- Set the speed control value to `128` , corresponding to approximately `50%` .
- Rotate forward for `3000 ms` .
- Stop the motor for `2000 ms` .
- Rotate in reverse for `3000 ms` .
- Stop the motor for `2000 ms` .
- Repeat the sequence continuously.

5.2 Program

In `src/main.c` , implement the program as follows:

```
#include "Arduino.h"
#include "dc_motor.h"

int main(void)
{
    setup();

    DCMotor_Init(GPI02, GPI03, GPI04, GPI05);

    while (1)
    {
        DCMotor_SetSpeed(128U);
        DCMotor_Forward();
        delay(3000U);

        DCMotor_Stop();
        delay(2000U);

        DCMotor_SetSpeed(128U);
        DCMotor_Reverse();
        delay(3000U);

        DCMotor_Stop();
        delay(2000U);
    }

    return 0;
}
```

Listing 14.5.1. DC motor control program using EduFramework

5.3 Program Explanation

`setup()` initializes the basic EduFramework components. Then, `DCMotor_Init(GPI02, GPI03, GPI04, GPI05)` configures the four control pins used by the DC Motor API. In the example TB6612FNG configuration, these pins correspond to `PWMA`, `AIN1`, `AIN2`, and `STBY`. Inside the main loop, `DCMotor_SetSpeed(128U)` sets the control level to approximately half of the `0..255` range. `DCMotor_Forward()` places the module in the forward configuration, and the motor remains in this state for `3000 ms`. Next, `DCMotor_Stop()` places the motor in the coast stop state for `2000 ms`. The program then applies the same speed level, calls `DCMotor_Reverse()` to change the direction configuration, and keeps the motor rotating in reverse for `3000 ms`. The motor is stopped for another `2000 ms` before the next cycle begins. The program flow can be summarized as:

Forward 50% → Stop → Reverse 50% → Stop → repeat.

5.4 Verification

Build and upload the program to the MaaZEDU Development Board. Apply external power to the motor driver module and observe the motor over several consecutive cycles.

Verify the following states:

- The motor rotates in one direction for approximately `3 s`.
- The motor coasts to a stop and remains stopped for approximately `2 s`.
- The motor rotates in the opposite direction for approximately `3 s`.
- The motor remains stopped for approximately `2 s` before the sequence repeats.

EXPECTED RESULT

The DC motor continuously repeats the sequence forward → stop → reverse → stop. Both rotation states use the same control value of 128, corresponding to approximately 50% of the DC Motor Device API control range. Each rotation state lasts approximately 3 s, and each stop state lasts approximately 2 s.

6 Extension

The main exercise uses one speed level and coast stopping to focus on basic motor control. The DC Motor Device API also provides functions for exploring additional motor-control states.

6.1 Changing the Speed Level

Replace the value 128 in the program with different values and observe the relative change in motor speed:

Value	Control Level	Observation
64	approximately 25%	Low rotation level.
128	approximately 50%	Level used in the main exercise.
191	approximately 75%	Higher rotation level.
255	100%	Maximum control level.

Extension: Apply the four values above one at a time while keeping the same rotation direction and observe the trend. RPM measurement is not required; the actual motor speed will be measured with an encoder in Lab 15.

6.2 Coast Stop and Short Brake

The TB6612FNG supports both coast stop and short-brake states [3]. In EduFramework:

- `DCMotor_Stop()` uses coast stopping;
- `DCMotor_Brake()` requests the short-brake state.

Syntax:

```
DCMotor_Stop();
DCMotor_Brake();
```

DCMotor_Brake

Stop the motor using the short-brake state of the H-bridge.

Syntax

```
DCMotor_Brake();
```

Return value

No return value.

Extension: Run the motor at the same speed level, then separately test `DCMotor_Stop()` and `DCMotor_Brake()`. Observe the difference between the two stopping methods without changing the hardware.

6.3 Standby Mode

The `STBY` pin of the TB6612FNG places the module in standby and disables the power outputs [3]. EduFramework provides `DCMotor_Standby()` to control this state.

DCMotor_Standby

Enable the motor driver module or place it in standby mode.

Syntax

```
DCMotor_Standby(standby);
```

Parameters

Parameter	Type / accepted value	Description
<code>standby</code>	<code>0</code> / non-zero	<code>0</code> enables the module; a non-zero value places the module in standby.

Return value

No return value.

Example:

```
DCMotor_Standby(1U);  
delay(2000U);  
DCMotor_Standby(0U);
```

Extension: Add a standby interval to the control sequence and verify that the motor is not driven while the module remains in standby.

7 References

- [1] Purdue University, *Lab 7: Implementing Actuators*, ME588 - Mechatronics, 2015.
- [2] Jonathan W. Valvano and Andreas Gerstlauer, *ECE445M/ECE380L.12 - Lecture 8*, The University of Texas at Austin, 2025.
- [3] Toshiba Electronic Devices & Storage Corporation, *TB6612FNG - Driver IC for Dual DC Motor*, TB6612FNG, 2026.
- [4] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet*, S32K1XX, Rev. 15, 2026.
- [5] MaaZEDU, *MaaZEDU Development Board Guide*, Rev. 1.0, 2025.
- [6] EduFramework, *DC Motor Device API*, EduFramework Source Code.